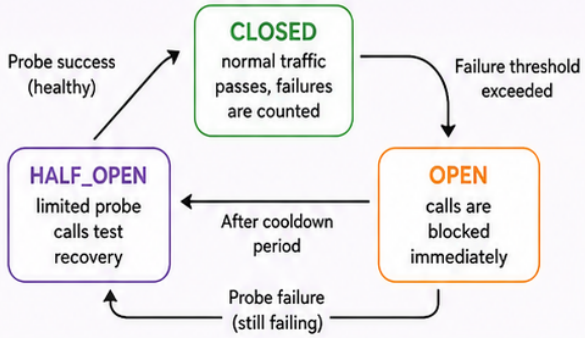
# Day 29: Circuit Breaker, Failure Thresholds, and Recovery Probing

*Preventing cascading failures by failing fast and restoring traffic safely.*

## 1. Why Circuit Breakers Matter

- Repeated calls to a failing dependency waste threads and increase latency
- Fail-fast behavior protects upstream capacity
- Helps stop cascading failures during incidents

## 2. State Machine



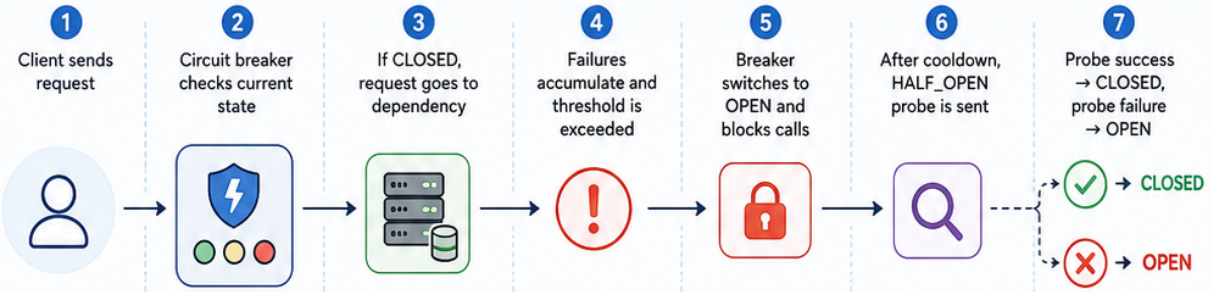

## 3. Failure Threshold & Cooldown

- Breaker opens after configured recent failures
- Cooldown window prevents retry storms
- Threshold and cooldown need careful tuning

## 4. Recovery & Fallback

- OPEN state returns fallback or fast error
- Successful HALF_OPEN probe closes breaker
- Failed probe reopens breaker and restarts cooldown

## 5. Sample Circuit Breaker Flow




## 6. Interview Takeaways

- Circuit breakers are a core anti-cascading-failure mechanism
- Fail-fast protects latency and thread pools
- Half-open probing enables controlled recovery
- Thresholds should be tuned using real data

## 7. Key Metrics

- Success
- Failure
- Blocked
- State transitions
- Open duration

**Circuit breakers protect system capacity by cutting off failing calls quickly and restoring traffic safely.**

# System Design - Day 29

Circuit Breaker, Failure Thresholds, and Recovery Probing

Prepared from Day-29 notes and sample implementation

## Overview from README.md

**Day 29: Circuit Breaker, Failure Thresholds, and Recovery Probing**

Day 29 focuses on preventing cascading failures when downstream dependencies are failing or slow.

It covers:
- circuit breaker fundamentals
- closed/open/half-open states
- failure threshold and cooldown windows
- probe requests for dependency recovery
- fallback and fail-fast behavior
- reliability trade-offs and tuning

In short, Day 29 is about cutting off repeated failing calls quickly and restoring traffic safely when a dependency recovers.

**Core ideas**

**Why circuit breakers matter**
- Repeated calls to a failing dependency waste threads and increase latency.
- Without protection, failures propagate upstream and can trigger wider outages.
- Circuit breakers fail fast during dependency incidents and protect service capacity.

**State machine model**
- `CLOSED`: normal mode; calls pass through and failures are counted.
- `OPEN`: calls are blocked immediately after failure threshold is exceeded.
- `HALF_OPEN`: after cooldown, limited probe calls test whether dependency recovered.

**Failure thresholds and cooldown**
- Breaker opens after a configured number of recent failures.
- Cooldown timer prevents immediate retry storms.
- After cooldown, a probe decides whether to close breaker or reopen.

**Recovery and fallback**
- In `OPEN`, system should return fallback/fast error instead of waiting on dependency timeouts.
- Successful probe in `HALF_OPEN` closes breaker and restores normal traffic.
- Failed probe reopens breaker and restarts cooldown.

**Trade-offs**
- Aggressive thresholds improve protection but can trip too often.
- Lenient thresholds reduce false trips but allow more failing traffic.
- Tuning depends on dependency SLO, latency budgets, and error budgets.

**Day-29 sample: Circuit Breaker Dependency Guard**

This repository includes a small circuit breaker sample in both Python and Java.

**Functional requirements**
- Wrap downstream calls with a circuit breaker
- Track failures and open after threshold
- Block calls while open

- Transition to half-open after cooldown ticks
- Close on successful probe or reopen on failed probe
- Report request outcomes and breaker state over time

**High-level components**
- `CircuitBreaker`: state machine and thresholds
- `DependencySimulator`: scripted success/failure outcomes
- `GuardedClient`: call wrapper with fallback behavior
- `Metrics`: counters for success, failure, blocked, and state transitions

**Data flow**
1. Client attempts downstream call through breaker
2. Breaker decides allow/block based on state
3. If allowed, dependency result updates breaker state
4. On threshold breach, breaker opens and starts cooldown
5. After cooldown, half-open probe determines recovery path

**Project structure**

```
Day-29/
README.md
circuit-breaker-sample/
python/
circuit_breaker.py
demo.py
java/
CircuitBreakerSystem.java
Main.java
```

**Run the sample**

**Python**

```
cd circuit-breaker-sample\python
python demo.py
```

**Java**

```
cd circuit-breaker-sample\java
javac Main.java CircuitBreakerSystem.java
java Main
```

**Interview takeaways**
- Circuit breakers are a core anti-cascading-failure mechanism
- Open-state fail-fast protects latency and thread pools
- Half-open probing enables controlled recovery
- Threshold and cooldown tuning must be data-driven
- Breakers complement retries, backpressure, and bulkheads

**Next improvements**
- Add rolling-window failure rate thresholds
- Add separate handling for timeout vs functional errors
- Add per-endpoint breakers and adaptive cooldown
- Add fallback response strategies by error class
- Add SLO-based alerting for breaker open duration

# Reference Implementation

Python and Java sample source files

### File: circuit-breaker-sample/python/demo.py

```
from circuit_breaker import CircuitBreaker, DependencySimulator, GuardedClient


def main() -> None:
    # False means dependency failure, True means success.
    scripted = [False, False, False, False, True, True, False, True]
    breaker = CircuitBreaker(failure_threshold=3, cooldown_ticks=2)
    dependency = DependencySimulator(scripted)
    client = GuardedClient(breaker, dependency)

    print("tick results")
    for tick in range(1, 11):
        outcome = client.request()
        print(
            f"  tick={tick} before={outcome.state_before.value} "
            f"allowed={outcome.allowed} result={outcome.result} "
            f"after={client.breaker.state.value}"
        )

    print("\nsummary")
    print(f"  success={client.success}")
    print(f"  failure={client.failure}")
    print(f"  blocked={client.blocked}")
    print(f"  final_state={client.breaker.state.value}")


if __name__ == "__main__":
    main()
```

### File: circuit-breaker-sample/python/circuit_breaker.py

```python
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class BreakerState(Enum):
    CLOSED = "CLOSED"
    OPEN = "OPEN"
    HALF_OPEN = "HALF_OPEN"


@dataclass(frozen=True)
class CallOutcome:
    state_before: BreakerState
    allowed: bool
    result: str


class CircuitBreaker:
    def __init__(self, failure_threshold: int, cooldown_ticks: int) -> None:
        if failure_threshold <= 0:
            raise ValueError("failure_threshold must be positive")
        if cooldown_ticks <= 0:
            raise ValueError("cooldown_ticks must be positive")

        self.failure_threshold = failure_threshold
        self.cooldown_ticks = cooldown_ticks
        self.state = BreakerState.CLOSED
        self._consecutive_failures = 0
        self._cooldown_remaining = 0

    def allow(self) -> bool:
        if self.state == BreakerState.OPEN:
            self._cooldown_remaining -= 1
            if self._cooldown_remaining <= 0:
                self.state = BreakerState.HALF_OPEN
                return True
            return False
        return True

    def on_success(self) -> None:
        self._consecutive_failures = 0
        self._cooldown_remaining = 0
        self.state = BreakerState.CLOSED

    def on_failure(self) -> None:
        if self.state == BreakerState.HALF_OPEN:
            self._trip_open()
            return

        self._consecutive_failures += 1
        if self._consecutive_failures >= self.failure_threshold:
            self._trip_open()

    def _trip_open(self) -> None:
        self.state = BreakerState.OPEN
        self._cooldown_remaining = self.cooldown_ticks
        self._consecutive_failures = 0


class DependencySimulator:
    def __init__(self, scripted_results: list[bool]) -> None:
        if not scripted_results:
            raise ValueError("scripted_results must be non-empty")
        self._results = scripted_results[:]
        self._index = 0

    def call(self) -> bool:
        if self._index >= len(self._results):
            return self._results[-1]
        outcome = self._results[self._index]
```

```
        self._index += 1
        return outcome


class GuardedClient:
    def __init__(self, breaker: CircuitBreaker, dependency: DependencySimulator) -> None:
        self.breaker = breaker
        self.dependency = dependency
        self.success = 0
        self.failure = 0
        self.blocked = 0

    def request(self) -> CallOutcome:
        before = self.breaker.state
        if not self.breaker.allow():
            self.blocked += 1
            return CallOutcome(state_before=before, allowed=False, result="BLOCKED")

        ok = self.dependency.call()
        if ok:
            self.breaker.on_success()
            self.success += 1
            return CallOutcome(state_before=before, allowed=True, result="SUCCESS")

        self.breaker.on_failure()
        self.failure += 1
        return CallOutcome(state_before=before, allowed=True, result="FAILURE")
```

### File: circuit-breaker-sample/java/Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Boolean> scripted = List.of(false, false, false, false, true, true, false, true);
        CircuitBreakerSystem.CircuitBreaker breaker = new CircuitBreakerSystem.CircuitBreaker(3, 2);
        CircuitBreakerSystem.DependencySimulator dependency = new CircuitBreakerSystem.DependencySimulator(scrip
        CircuitBreakerSystem.GuardedClient client = new CircuitBreakerSystem.GuardedClient(breaker, dependency);

        System.out.println("tick results");
        for (int tick = 1; tick <= 10; tick += 1) {
            CircuitBreakerSystem.CallOutcome outcome = client.request();
            System.out.println(
                "  tick="
                    + tick
                    + " before="
                    + outcome.getStateBefore()
                    + " allowed="
                    + outcome.isAllowed()
                    + " result="
                    + outcome.getResult()
                    + " after="
                    + client.getBreaker().getState()
            );
        }

        System.out.println("\nsummary");
        System.out.println("  success=" + client.getSuccess());
        System.out.println("  failure=" + client.getFailure());
        System.out.println("  blocked=" + client.getBlocked());
        System.out.println("  final_state=" + client.getBreaker().getState());
    }
}
```

### File: circuit-breaker-sample/java/CircuitBreakerSystem.java

```
import java.util.ArrayList;
import java.util.List;

public class CircuitBreakerSystem {
    public enum BreakerState {
        CLOSED,
        OPEN,
        HALF_OPEN
    }

    public static final class CallOutcome {
        private final BreakerState stateBefore;
        private final boolean allowed;
        private final String result;

        public CallOutcome(BreakerState stateBefore, boolean allowed, String result) {
            this.stateBefore = stateBefore;
            this.allowed = allowed;
            this.result = result;
        }

        public BreakerState getStateBefore() {
            return stateBefore;
        }

        public boolean isAllowed() {
            return allowed;
        }

        public String getResult() {
            return result;
        }
    }

    public static final class CircuitBreaker {
        private final int failureThreshold;
        private final int cooldownTicks;
        private BreakerState state = BreakerState.CLOSED;
        private int consecutiveFailures = 0;
        private int cooldownRemaining = 0;

        public CircuitBreaker(int failureThreshold, int cooldownTicks) {
            if (failureThreshold <= 0) {
                throw new IllegalArgumentException("failureThreshold must be positive");
            }
            if (cooldownTicks <= 0) {
                throw new IllegalArgumentException("cooldownTicks must be positive");
            }
            this.failureThreshold = failureThreshold;
            this.cooldownTicks = cooldownTicks;
        }

        public boolean allow() {
            if (state == BreakerState.OPEN) {
                cooldownRemaining -= 1;
                if (cooldownRemaining <= 0) {
                    state = BreakerState.HALF_OPEN;
                    return true;
                }
                return false;
            }
            return true;
        }

        public void onSuccess() {
            consecutiveFailures = 0;
            cooldownRemaining = 0;
            state = BreakerState.CLOSED;
        }

        public void onFailure() {
            if (state == BreakerState.HALF_OPEN) {
```

```
            tripOpen();
            return;
        }

        consecutiveFailures += 1;
        if (consecutiveFailures >= failureThreshold) {
            tripOpen();
        }
    }

    private void tripOpen() {
        state = BreakerState.OPEN;
        cooldownRemaining = cooldownTicks;
        consecutiveFailures = 0;
    }

    public BreakerState getState() {
        return state;
    }
}

public static final class DependencySimulator {
    private final List<Boolean> results;
    private int index = 0;

    public DependencySimulator(List<Boolean> scriptedResults) {
        if (scriptedResults == null || scriptedResults.isEmpty()) {
            throw new IllegalArgumentException("scriptedResults must be non-empty");
        }
        this.results = new ArrayList<>(scriptedResults);
    }

    public boolean call() {
        if (index >= results.size()) {
            return results.get(results.size() - 1);
        }
        boolean outcome = results.get(index);
        index += 1;
        return outcome;
    }
}

public static final class GuardedClient {
    private final CircuitBreaker breaker;
    private final DependencySimulator dependency;
    private int success = 0;
    private int failure = 0;
    private int blocked = 0;

    public GuardedClient(CircuitBreaker breaker, DependencySimulator dependency) {
        this.breaker = breaker;
        this.dependency = dependency;
    }

    public CallOutcome request() {
        BreakerState before = breaker.getState();
        if (!breaker.allow()) {
            blocked += 1;
            return new CallOutcome(before, false, "BLOCKED");
        }

        boolean ok = dependency.call();
        if (ok) {
            breaker.onSuccess();
            success += 1;
            return new CallOutcome(before, true, "SUCCESS");
        }

        breaker.onFailure();
        failure += 1;
        return new CallOutcome(before, true, "FAILURE");
    }
    public CircuitBreaker getBreaker() {
```

```
            return breaker;
        }

        public int getSuccess() {
            return success;
        }

        public int getFailure() {
            return failure;
        }

        public int getBlocked() {
            return blocked;
        }
    }
}
```